Babeş-Bolyai University
Faculty of Mathematics and Computer Science
Master's Programme in High Performance Computing

GPU-Accelerated Learned Spatial Indexing: Extending LISA to an End-to-End GPU Pipeline

Master's Dissertation

Author: Darius-Ştefan Jurjiu-Tandrău
Supervisor: Prof. Dr. Virginia Niculescu

Cluj-Napoca
June 2026

Abstract

Learned index structures replace the comparison-based traversal of classical indexes with machine-learned models that predict where a key is stored. LISA, a learned index for spatial data introduced at SIGMOD 2020, demonstrated that this idea extends to multi-dimensional points, but its reference implementation is sequential and CPU-bound. This dissertation investigates how much of the LISA pipeline can be moved to a GPU, and what that is worth in practice.

The pipeline is rebuilt stage by stage in CuPy. Grid partitioning and the locality-preserving mapping collapse into single global sorts, exploiting the fact that LISA's mapping assigns disjoint value ranges to distinct cells. The training of all per-column piecewise-linear models is restructured as a single batched Newton optimisation over padded tensors. Range queries are reformulated as a flat candidate index processed by broadcasted filtering and scatter-add; kNN queries combine GPU lattice-regression inference with a batched top-k selection over ragged candidate sets; and bulk insertions and deletions are recast as stable sorts over shard identifiers. One further direction is evaluated and reported as a negative result: a monotonic multilayer perceptron as an alternative local model, which trains quickly but fits far worse than the piecewise-linear baseline on most workloads.

The implementation is evaluated on uniform, clustered, and real-world (GeoNames) datasets of up to 10^7 points, on two GPUs: an NVIDIA A100 and an NVIDIA L4. Against the reference CPU implementation, GPU batch range queries run up to nearly 100× faster, model training up to 26× faster, and batched insertions up to 26× faster. The GPU range-query path also outperforms classical CPU baselines, processing batches 10–17× faster than a bulk-loaded R-tree and 15–46× faster than a quad-tree. For kNN queries, however, a single-core `cKDTree` remains orders of magnitude faster at the evaluated scales, and an accuracy analysis shows that the radius-driven kNN design returns approximate neighbour sets for over half the queries on uniform data — both findings are measured and traced to their causes rather than explained away. The dissertation closes with a discussion of the conditions under which GPU-resident learned spatial indexes are the right tool, and of the work still needed to make them general.

Contents

1. Introduction

1.1 Motivation

Spatial data is being produced at rates that strain the index structures databases have relied on for four decades. Location-tagged events, GPS traces, point-of-interest catalogues and sensor feeds are all, at bottom, large collections of low-dimensional points that must answer the same two questions quickly: *which points fall in this box?* and *which points are nearest to this one?* The standard answer since the 1980s has been the R-tree [7] and its relatives — the quad tree [8] and the k-d tree [9] — which organise space hierarchically and prune irrelevant regions during search. These structures were designed under assumptions that no longer describe the computing landscape well: data small enough for pointer-based hierarchies to stay cheap, and processors that execute one comparison at a time. Both assumptions have eroded, from two different directions.

From the algorithmic direction, the *learned index* reframed what an index is. Kraska et al. [1] observed that a B-tree is, functionally, a model that maps a key to a position in a sorted array — an approximation of the data's cumulative distribution — and that a small learned regression model can perform the same mapping; their measurements report lookups up to 70% faster than a cache-optimised B-Tree at an order of magnitude less memory. Carrying this idea into the spatial domain is not straightforward, because multi-dimensional keys have no natural sorted order for a model to learn; LISA [2] solved this by *constructing* such an order. It replaces the R-tree's hierarchy with a learned pipeline: a grid partition of space, a monotone function that maps points to one dimension, and a family of small piecewise-linear models that predict the disk shard in which a mapped key resides. On large datasets LISA answers range and kNN queries with less storage and less I/O than an R-tree.

From the hardware direction, the processor itself has changed shape. The arithmetic throughput of GPUs has grown far faster than that of CPUs over the same period, yet tree-shaped index structures profit little from it: pointer chasing, data-dependent branching and per-query work imbalance all conflict with the GPU's lock-step SIMT execution model [5]. Learned indexes are a far better fit, since they replace traversal with arithmetic — sorts, prefix sums, and dense model evaluation — which is precisely the workload GPUs are built for. Liu et al. demonstrated this for one-dimensional learned indexes with the G-Learned Index [4], reporting speedups of two orders of magnitude over CPU learned indexes. Spatial learned indexes, however, have remained CPU-bound: the recent survey by Al-Mamun et al. [6] catalogues dozens of proposals, none of which targets GPU execution — the survey itself treats hardware-platform aspects as outside its scope.

The two directions therefore point at each other: spatial learned indexes exist but ignore the GPU, and GPU learned indexes exist but cannot answer spatial queries. This dissertation sits in that gap. It asks a concrete question: *if the entire LISA pipeline — construction, model training, range queries, kNN queries, and updates — is re-expressed in GPU-friendly form, which stages benefit, by how much, and which do not?* Both halves of the question carry equal weight here. Several results in this thesis are negative or mixed, and measuring and explaining them is as much a part of the contribution as the speedups.

1.2 Contributions

The dissertation makes the following contributions, listed roughly in decreasing order of novelty.

1. **Batched GPU training of LISA's local models** (Section 4.2). The reference implementation trains its M piecewise-linear models one at a time, each with its own Newton iteration and line search. We restructure the whole problem as one batched optimisation over a padded $(M, n_{\max}, \sigma)$ tensor, with a batched linear solve, a vectorised ten-point line search, and per-model best-so-far tracking under an activity mask. To our knowledge no published learned-index implementation trains its local models this way. Measured end to end, training all 64 models takes 513 s on the CPU and 19 s on the GPU at $N{=}10^6$ (uniform data) — a 26.5× speedup — and the GPU path trains $N{=}10^7$ in under three minutes, where the sequential CPU baseline is no longer practical to run at all.

2. **A fully GPU-resident range-query path** (Section 4.4), built around a flat (query, point) candidate index constructed with `repeat`/`cumsum` primitives, a broadcasted box filter and a scatter-add reduction, with memory-bounded chunking. At $N{=}10^7$ it answers 1,000 range queries in 35 ms against 3.5 s for the CPU implementation (100×) and 434 ms for a bulk-loaded R-tree (12×).

3. **A sort-collapsed GPU build** (Section 4.1). LISA's mapping assigns provably disjoint value ranges to distinct grid cells, so the reference implementation's T per-cell sorts are equivalent to one global sort of all N mapped values — which, together with a vectorised mapping evaluation, turns the entire layout construction into a handful of large GPU kernels. Partitioning runs up to 573× faster at $N{=}10^7$.

4. **A batched GPU kNN path** (Section 4.5) combining lattice-regression radius prediction (inference reformulated as a dense gather), iterative box growth, and a slot-assignment trick that performs top-k selection over ragged per-query candidate sets without any per-query loop. It comes out four to seven times as fast as the CPU implementation, but it does not catch a `cKDTree` at any scale we tried — and an

accuracy analysis shows that the radius stopping rule makes the returned neighbour sets approximate for over half the queries on uniform data. Chapter 6 has the numbers and the reasons.

5. **Batched dynamic updates as sort problems** (Section 4.6). Insertion and deletion are reframed from per-point page manipulation into stable sorts keyed by predicted shard, followed by histogram and prefix-sum reconstruction of the shard offsets. Bulk-insert throughput reaches 17 million points per second at $N{=}10^6$ and 3.6 million at $N{=}10^7$.

6. **A negative result, reported as one** (Section 6.7): a monotonic MLP local model, although somewhat cheaper to train than the batched piecewise trainer, fits dramatically worse on almost all workloads and is not integrated into the query path. It is kept in the thesis because knowing that this direction does not (yet) work is itself useful.

7. **An evaluation across three datasets, four classical baselines and two GPUs**, with seeds and protocol documented for reproducibility.

Throughout, we are explicit about what the comparison baselines are. In particular, the CPU LISA used for speedup figures is the publicly available Python implementation, updated to run under Python 3; the native implementation behind the original paper's experiments was never released. CPU-versus-GPU ratios should therefore be read as "GPU versus an optimised NumPy implementation," not versus hand-tuned native code. This and other threats to validity are collected in Section 5.6.

1.3 Outline

Chapter 2 provides the background in three parts: classical spatial indexing (R-trees, quad trees, k-d trees, space-filling curves), the learned-index family from the original one-dimensional proposal through its multi-dimensional descendants, and the GPU execution model — the three platform properties that dictate every design decision made later. It closes by locating the gap this thesis occupies.

Chapter 3 describes LISA itself: what kind of index it is, the four components it consists of (mapping function, shard prediction, local models, and the lattice regression used for kNN), the publicly available reference implementation this work builds on, the flat storage layout we add to it, the reference training algorithm, and the exact index parameters used throughout the evaluation.

Chapter 4 is the technical core: stage by stage, how each part of the pipeline was reformulated for the GPU — the sort-collapsed build, the batched Newton trainer (the

thesis's main methodological contribution, including its three documented departures from the reference optimiser), batched shard prediction, the flat-candidate range query, the kNN path with its ragged top-k selection, dynamic updates recast as sort problems, the exploratory monotonic MLP, and the correctness test suite that runs every GPU code path without a GPU.

Chapter 5 documents the experimental conditions: the two evaluation machines (A100 and L4), the three datasets (uniform, clustered synthetic, and the real-world GeoNames gazetteer), the query workloads with their seeds, the measurement protocol with its two stated caveats, the four classical baselines, and a consolidated list of threats to validity.

Chapter 6 presents the measurements in the pipeline's order — build, training, shard prediction, range queries (including the comparison against R-tree and quad-tree), kNN (including both the speed comparison against `cKDTree` and brute force, and an accuracy analysis of the radius stopping rule), dynamic operations, and the monotonic MLP — followed by the cross-GPU comparison and a summary of where GPU execution does not pay.

Chapter 7 collects the limitations with their corresponding future-work directions, and Chapter 8 summarises what was established, for and against.

2. Background and Related Work

2.1 Spatial indexing

A spatial index organises a set of points $P \subset \mathbb{R}^d$ so that queries touch only a small fraction of the data. The two query types this thesis evaluates are the *axis-aligned range query* — given a box $[l_1, u_1] \times \cdots \times [l_d, u_d]$, return (or count) the points inside — and the k *nearest neighbours* (kNN) query — given a point q, return the k points minimising Euclidean distance to q.

R-trees. The R-tree [7] groups nearby points into minimum bounding rectangles (MBRs) and nests these rectangles into a balanced tree, every node sized to a disk page. A range query descends from the root, visiting exactly those subtrees whose rectangle intersects the query region; a subtree whose MBR misses the box is pruned with all its contents. The structure's quality therefore depends on how well the rectangles separate: when siblings overlap, a query must descend several branches for the same region, and query cost grows with the overlap. How the tree is built matters as much as how it is searched. Inserting points one at a time forces heuristic choices (which subtree to enlarge, how to split a full node) that accumulate overlap; for static datasets, bulk loading avoids this by constructing the tree bottom-up from pre-sorted data. The variant we benchmark against uses Sort-Tile-Recursive (STR) bulk loading [10]: sort by one coordinate, cut into vertical slices, sort each slice by the other coordinate, and pack tiles into leaves — producing tight, low-overlap rectangles at a cost close to that of sorting. The R-tree's weaknesses at scale are equally well documented: the tree stores an MBR and pointers per node on top of the data itself, page utilisation degrades under updates, and overlap grows back as insertions accumulate.

Quad trees and k-d trees. The quad tree [8] recursively subdivides space into four quadrants — at the data points themselves in Finkel and Bentley's original point quad tree, or at fixed cell midpoints in the region variants common in practice (our baseline, `pyqtree`, is of the latter kind) — until each leaf holds few enough points. The region variant's decomposition is fixed by geometry rather than by the data, which keeps the structure simple but lets skewed data drive it deep and unbalanced. The k-d tree [9] splits along alternating coordinate axes; with balanced split rules (medians or sliding midpoints in modern implementations), the tree stays shallow regardless of the distribution and gives logarithmic point lookups. Its kNN search is a small branch-and-bound: descend to the query's leaf, then unwind, visiting a sibling branch only if the sphere through the current k-th-best neighbour crosses the splitting plane — on low-dimensional data this prunes almost everything, and the cost grows only gently with N. This is worth stating concretely because it sets the bar for Section 6.5: SciPy's `cKDTree` [17], a C implementation of this

approach, answers a 10-NN query over 10^7 two-dimensional points in a few microseconds. In low dimensions, the k-d tree is the strongest practical kNN baseline available off the shelf; its pruning argument collapses only as dimensionality grows, when the bounding sphere crosses nearly every splitting plane.

Space-filling curves. An alternative line of work linearises space along a fixed curve and reuses one-dimensional machinery. The Z-order curve interleaves the bits of the coordinates, so that points close in space usually receive close one-dimensional keys; the Hilbert curve improves the locality guarantee at the cost of a more involved encoding. Both share a structural weakness: locality is preserved only *usually* — the curve must occasionally jump across the domain, so a compact spatial region can map to several distant key ranges, and a one-dimensional range scan over the keys picks up points far outside the query region. The ZM index [11] combines a Z-order curve with a learned one-dimensional index over the resulting keys. LISA's mapping function (next chapter) is in the same spirit — reduce d dimensions to one — but differs in two ways that matter: the mapping is constructed from the *data distribution* (an equi-depth grid) rather than from a fixed geometric curve, and it is paired with models that *generate* the storage layout rather than approximate an existing one.

2.2 Learned indexes

Kraska et al. [1] framed an index as a model of the cumulative distribution function (CDF) of the keys: if $F(x)$ is the fraction of keys $\leq x$, then $N \cdot F(x)$ is the position of x in the sorted array, and a model of F with bounded error replaces a B-tree's traversal with a prediction plus a short local search. Their recursive model index (RMI) implements this with a small hierarchy of regressors: a root model maps the key to one of several second-stage models, each responsible for a slice of the key space; the final model predicts a position, and the known worst-case prediction error bounds the local search that follows. The appeal is twofold — the model is orders of magnitude smaller than the tree it replaces, and a lookup is a few multiply-adds instead of a pointer chase through cache-missing nodes. The original RMI is static; ALEX [12] made the idea updatable in one dimension by storing keys in gapped arrays that absorb insertions near their predicted positions, splitting and retraining nodes as regions fill — a useful reference point for the update discussion in Chapter 7.

Extending the idea beyond one dimension is not mechanical, because multi-dimensional keys have no natural total order: there is no single CDF to learn, and the queries that matter — boxes, neighbourhoods — constrain every coordinate at once. The proposals that followed split roughly into two strategies. One learns a *layout*: Flood [3], for instance, lays a grid over $d-1$ of the d dimensions and keeps each cell's points sorted by the remaining

dimension; the choice of that sort dimension and the number of partitions per gridded dimension are tuned by a cost model calibrated on the data and a sample query workload, and a range query visits exactly the cells its rectangle intersects, refining within each cell along the sorted dimension. The other learns a *projection to one dimension* and reuses one-dimensional machinery on the result — the ZM index above, and LISA. The survey by Al-Mamun et al. [6] maps what is by now a large family of such proposals across both strategies. LISA [2] is among the most complete in either: it supports range queries, kNN *and* updates over disk-resident data, which is why it was chosen as the basis of this work. Section 3.1 describes it in detail.

2.3 GPUs and the SIMT execution model

A modern GPU exposes thousands of arithmetic lanes organised into warps of 32 threads that execute in lock step, scheduled across on the order of a hundred streaming multiprocessors. Three properties of the platform shape every design decision in Chapter 4:

Throughput over latency. A single GPU thread is slow, and a read from device memory costs hundreds of cycles; the hardware pays off only when tens of thousands of threads are in flight, because the scheduler hides memory latency by running other warps while one waits. Two consequences follow. First, control-flow divergence is expensive: when threads within a warp branch differently, the warp executes both paths in sequence with the non-participating lanes masked off, so per-item decisions are best converted into arithmetic over flags. Second, memory access patterns matter as much as operation counts: a warp's loads from adjacent addresses coalesce into few wide transactions, while scattered loads waste most of every cache line fetched. Per-query tree descent gives each thread divergent control flow *and* scattered reads — the worst case on both axes — whereas batched primitives — `sort`, `cumsum`, `searchsorted`, histograms, gather/scatter, dense matrix products — keep all lanes busy over contiguous data.

Kernel-launch and transfer overhead. Every operation is a kernel launch costing a few microseconds regardless of how little work it carries, and host–device transfers cross PCIe with both latency and bandwidth far below on-device memory. The practical rules are to batch work into few large kernels rather than many small ones, to keep data resident on the device across operations, and to ship only control decisions back to the host. Workloads below roughly 10^5 elements rarely amortise these fixed costs — something our measurements reproduce clearly (Section 6.9).

Uniformity. Kernels want rectangular data and identical per-item work. Ragged, data-dependent structures (one variable-length list per query, early exits, per-item retries) must be reshaped: padded into rectangular tensors with masks, or flattened into single long arrays with offset tables that record where each item's slice begins. Much of Chapter 4 is

exactly this reshaping, and the recurring cost it accepts — computing results for finished items and discarding them — is the standing price of uniformity.

We implement against CuPy [13], which exposes the NumPy API over CUDA, and PyTorch [14] for the one stage that benefits from automatic differentiation (the monotonic MLP). The brute-force GPU kNN baseline computes exact pairwise distances and top-k with PyTorch (Section 5.5).

2.4 GPU-accelerated indexes

Porting classical spatial structures to the GPU has been tried, and the results illustrate the architectural mismatch concretely. Kim et al. [5] studied parallel traversal of R-trees and their variants on GPUs and found the structures ill-matched to the platform: the irregular traversal pattern of range queries makes it hard to utilise the many-core architecture. Their remedies are instructive — assigning subtrees to multiprocessors whose cores cooperate on node navigation, and restructuring the search to avoid non-sequential access to tree nodes — because both amount to forcing regularity onto an irregular structure. The lesson carried into this thesis is that the more profitable route is not parallelising tree descent but replacing it with batch-parallel arithmetic.

On the learned-index side, the most direct predecessor of this work is the G-Learned Index of Liu et al. [4], a GPU adaptation of learned indexing for one-dimensional keys. It accelerates one-dimensional learned lookups by restructuring them into exactly the workload shapes the GPU favours — large parallel sorts on the construction side and batched model evaluations on the query side — and reports an average 174× speedup over a state-of-the-art CPU learned index. Two things make it the natural predecessor of this work and at the same time insufficient for it. First, its central observation — that a learned index's execution time is dominated by sorting and dense model evaluation, two operations that map almost perfectly onto GPU execution — predicts what we observe for LISA's build phase, where speedups reach two orders of magnitude. Second, it is strictly one-dimensional: its keys have a total order, its queries are key lookups and key ranges, and nothing in it addresses the parts that make the spatial case hard — the dimension-reducing mapping, multi-stage queries with exact geometric filtering, radius-driven kNN, or spatial updates.

What is missing from the literature, and what this thesis provides, is a GPU treatment of a *complete spatial* learned index: not just the lookup function but training, multi-stage spatial queries with exact filtering, and updates — together with measurements of where the GPU does *not* help.

3. LISA and the CPU Port

3.1 The LISA pipeline

LISA — the Learned Index Structure for Spatial dAta, introduced by Li et al. at SIGMOD 2020 [2] — is a learned index for disk-resident multi-dimensional point data. It is designed as a replacement for the R-tree in exactly the R-tree's home setting: large spatial datasets answering range and kNN queries, with support for insertions and deletions, where the costs that matter are storage and the number of disk pages touched per query. On both metrics the original paper reports substantial improvements over R-trees and other alternatives.

Its central idea resolves the difficulty raised in Chapter 2: a learned index needs a sorted one-dimensional order to model, and spatial data has none — so LISA *manufactures* one, and then uses the machine-learned models twice. First, the data space is reduced to one dimension by a mapping built from the data's own distribution, and the points are sorted by their mapped values. Local models are then trained to predict where in this sorted order a mapped value falls, and the predictions are used to *generate* the physical storage layout: points the models place together are stored together, in fixed-size groups called shards. Second, at query time, the *same* models answer queries against that layout — a query maps its search region to the one-dimensional domain and asks the models which shards could contain answers. Because one function family both shelves the data and looks it up, predictions are self-consistent: a model error displaces a point and the search for it by the same amount, costing scan width rather than correctness, with a final exact geometric filter guaranteeing the answers. The pipeline can be held in one line: *sort by a learned-friendly key, learn the sorted order, cut it into shards, and answer queries by predicting between which two shards the answers lie.*

Concretely, LISA builds this layout for N points in $[0, V]^d$ through four components, which the original paper names the mapping function $\mathcal{M}$, the shard prediction function $\mathcal{SP}$, the local models, and (for kNN) a lattice regression model $\mathcal{LR}$. We describe each as implemented in the code base used here; the parameter symbols recur throughout the thesis.

Grid partitioning. The data space is cut into a grid of T^d cells, with cell borders chosen so that cells receive equal point counts (an equi-depth partition, computed by sorting the data along each of the first $d-1$ dimensions recursively). With our settings (T=50, d=2) this yields 50 vertical stripes, each holding $N/50$ points.

Mapping. Each point $\mathbf{x}$ in cell i is mapped to a scalar

$$\mathcal{M}(\mathbf{x}) \;=\; i\,T \;+\; \frac{x_d}{V}\,(T-1) \;+\; \eta\,\frac{\mu(\mathbf{x})}{\mu(C_i)}, \tag{3.1}$$

where x_d is the last coordinate, $\mu(\mathbf{x})$ is the volume of the sub-rectangle between the cell's lower corner and $\mathbf{x}$ in the first $d-1$ coordinates, $\mu(C_i)$ is the cell's measure, and η is a small weight (0.01 here). The term iT guarantees that distinct cells occupy *disjoint value ranges* — the property that later enables the single-global-sort optimisation on the GPU. Within a cell, the mapping is monotone in each coordinate, so a query box's corners bound the mapped values of its contents.

Columns, local models, shards. The mapped values of all points are sorted, then divided into M contiguous *columns* of roughly N/M points (M=64 here). For each column, LISA trains a small monotone model

$$f(x) \;=\; \sum_{j=1}^{\sigma} \alpha_j \cdot \max(0,\, x-\beta_j), \qquad \sigma = 50, \tag{3.2}$$

that regresses a mapped value onto its rank within the column. Monotonicity is enforced through the constraint that all prefix sums of α are non-negative with strictly increasing β; training minimises the sum of squared rank errors (Section 3.3 details the optimiser). Dividing the predicted rank by a page size p (100 points) yields a *shard* identifier. The important twist is that LISA uses the trained model itself to *generate* the physical layout: points predicted into the same shard are stored contiguously, in one or more pages. A query for mapped value x therefore evaluates f, divides by p, and lands directly on the right shard — no tree, no binary search over the data.

Range queries. A box query runs in three steps: *map*, *scan*, *filter*. First, both corners of the box are mapped through $\mathcal{M}$ and their shards predicted. This is where monotonicity becomes essential: within a cell the mapping is monotone in every coordinate, so any point inside the box has a mapped value between the two corner values — and shard prediction being monotone in turn, that point is stored somewhere between the two corner shards. The shard interval $[\,\text{shard(low corner)}, \text{shard(high corner)}\,]$ is therefore a provable superset of the answers, located at the cost of two model evaluations and two lookups into the flat layout. Second, every point stored in that interval — the *candidates* — is scanned. Third, each candidate's actual coordinates are compared against the box, and only true members survive this exact filter. The superset is a generous one: within a cell the mapping is dominated by the last coordinate, so the interval admits every point whose last coordinate is in range, regardless of where it sits along the other axes; and a box spanning several cells sweeps the intermediate cells into candidacy in their entirety. The original paper trims this with a per-cell overlap analysis. We kept the simpler corner-to-corner interval,

on both CPU and GPU paths, so the two backends always perform identical work; the price is extra filtering — candidate volume, not answer volume, is what the query pays for — and correctness is never at stake, because the final filter is exact (Section 6.4).

kNN queries. LISA converts kNN to range queries: a lattice regression model $\mathcal{LR}$ [15] – a smooth interpolant over a regular grid of anchor values, fitted with a Laplacian smoothness prior – predicts a search radius r for the query point, a box of half-width r is searched, and if fewer than k neighbours are found the radius is doubled and the search retried.

Updates. Because shards are generated by a monotone model, inserting a point means predicting its shard and appending to that shard's last page, splitting if full; deletion removes a point from its page.

3.2 The reference implementation and the flat layout

The starting point is the publicly available Python implementation of LISA, written for Python 2.7 with NumPy and SciPy. Getting it to run on a current software stack required only mechanical syntax updates (print statements, integer-division semantics, `cPickle`); no algorithmic behaviour was changed. This updated reference, running under NumPy, is the CPU baseline for every comparison in the thesis. As noted in the introduction, the native implementation used for the original paper's published experiments is not available, so a NumPy baseline is the strongest faithful one obtainable.

One structural element was added on top of the reference, shared by both backends: a *flat layout*. The reference code stores pages in a Python list and walks per-shard page metadata at query time — acceptable on a CPU, ill-suited to a GPU. We observe that pages are generated in shard order, so concatenating them yields an (N, d) array `all_points` in which every shard owns one contiguous slice, describable by two integer arrays `shard_point_starts` and `shard_point_ends`. This is the same trick as a CSR sparse matrix's `indptr` array, and it reduces "find the candidate points for a shard interval" to two array lookups. The flat layout is built once after page generation and is reused by range queries, kNN and the dynamic operations on both CPU and GPU.

3.3 The CPU training loop

The reference trainer optimises each column's (α, β) with up to 200 iterations of a damped Newton method. One iteration: build the ReLU design matrix $A_{ij} = \max(0, x_i - \beta_j)$; solve the $\sigma \times \sigma$ normal equations for the optimal α given β (guarded by an explicit condition-number check, falling back to a monotone re-projection when the unconstrained solution violates the constraints); form the gradient and a Gauss–Newton-style Hessian in

β; solve for the step; then evaluate *ten* candidate learning rates and keep the best valid one. If no learning rate improves the loss, the model falls back to a gradient step, and failing that, exits early.

Two properties matter for what follows. First, the per-model problem is small ($\sigma = 50$ unknowns, a few times 10^4 residuals) — far too small to occupy a GPU on its own. Second, the control flow is heavily data-dependent: different models converge at different iterations, take different fallback paths, and exit at different times. The contribution of Section 4.2 is a restructuring that preserves the optimiser's behaviour closely enough to match its training losses while eliminating every data-dependent branch.

3.4 Index parameters used throughout

All experiments use the same index configuration: $T = 50$ grid partitions per dimension, $M = 64$ piecewise models, $\sigma = 50$ ReLU components per model, page size $p = 100$, and $\eta = 0.01$, over the domain $[0, 10^4]^2$. These are scaled down from the reference configuration — the paper evaluates two-dimensional data with $T = 240$, and the authors' released implementation defaults to $M = 1024$ models with $\sigma = 100$ — so that the sequential CPU trainer remains runnable within benchmark time budgets at $N \leq 10^6$. The parameter count of the learned part of the index is pleasantly small either way: $2M\sigma = 6{,}400$ doubles, or about 51 KB, plus per-shard offset tables (about 1.6 MB at $N{=}10^7$) — versus tens to hundreds of megabytes for the R-tree over the same data (Section 6.4).

4. GPU Design and Implementation

This chapter describes how each stage of the pipeline was made to run on the GPU. Figure 4.1 shows the stages and where each executes. A few design rules applied everywhere and are worth stating once:

- *Batch or do not bother.* No stage processes one query, one model or one point at a time. Where the problem is naturally ragged — variable-length columns, variable-size candidate sets — it is padded into rectangular tensors with masks, or flattened into a single long array with offset tables.
- *Replace control flow with arithmetic.* Early exits become activity masks; retry loops become vectorised "grow and re-test" rounds over only the unfinished queries; per-model conditional fallbacks become unconditional regularisation.
- *Move data once.* Model parameters and the flat point layout are transferred to the GPU a single time and reused across calls; benchmarks time transfers separately from compute so the steady-state and cold-start views are both visible.
- *Keep one source of truth.* The GPU modules are written against an `xp` array-module variable that binds to CuPy when a GPU is present and to NumPy otherwise. The same code therefore runs — slowly but bit-compatibly — on a laptop, which is what makes the correctness test suite (Section 4.8) executable without CUDA hardware.

4.1 Build phase: partition, mapping, global sort

Everything downstream consumes the build stages' output, so we describe them first; they are also where the GPU's advantage turns out to be largest.

The partition stage sorts the data along each of the first $d-1$ dimensions recursively; for d=2 that is one N-element sort, executed with `cp.argsort`. The interesting stage is mapping-plus-sort. The reference code computes Equation (3.1) cell by cell and sorts each cell's points separately — T small dependent sorts. But the iT term in the mapping gives different cells provably disjoint output ranges, so sorting each cell independently is equivalent to sorting *all* mapped values at once. The GPU version therefore evaluates the mapping for all N points with one vectorised expression (a gather of per-cell borders and measures followed by elementwise arithmetic) and runs a single global `argsort`. Two PCIe transfers bracket the whole build: raw points in, sorted points and mappings out.

This restructuring is also why build-phase speedups in Chapter 6 are so large (94× partition, 18× mapping-plus-sort at N=10^6, growing to 449× and 21× at 10^7): a single large radix-style sort is close to the GPU's best case, and the restructured pipeline contains

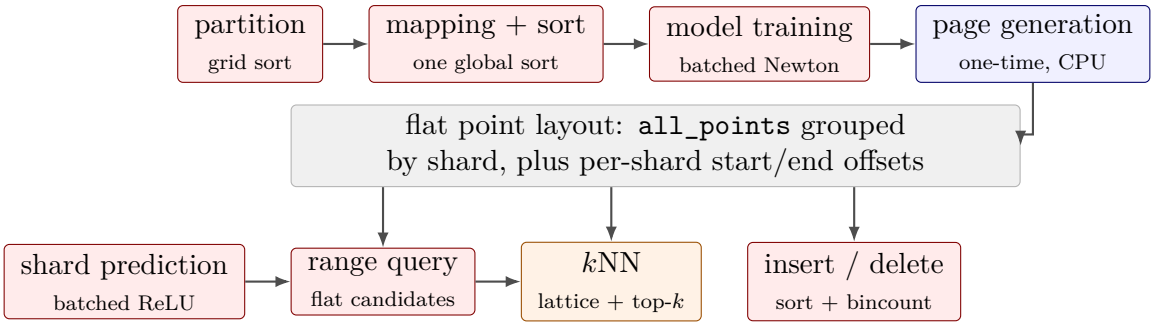

Figure 4.1: The pipeline as implemented. The build chain (top) ends with page generation, which produces the flat point layout — the array all query-side operations read (and, for insert/delete, rewrite). Shard prediction feeds the range query, which in turn is the engine inside kNN. Red stages run on the GPU, blue on the CPU, orange partly on each (the kNN radius model is trained on the CPU, evaluated on the GPU).

essentially nothing else.

4.2 Batched training of all local models

Training is where the largest share of build time goes — about 510 s at $N{=}10^6$ on the CPU (uniform data), against 0.35 s for everything else combined — and it is the stage whose GPU formulation required the most care.

4.2.1 What exactly is being optimised

It is worth restating the problem concretely before parallelising it. One column holds n points sorted by mapped value; call those values $x_1 \le x_2 \le \cdots \le x_n$. The model's job is to return each value's position: $f(x_i) \approx i$. LISA's model is a sum of $\sigma = 50$ shifted ramps,

$$f(x) \;=\; \sum_{j=1}^{\sigma} \alpha_j \cdot \max(0,\, x - \beta_j), \tag{4.1}$$

and the two parameter vectors have plain geometric meanings. Each β_j is a *knot*: an input value at which the curve is allowed to change slope. Each α_j says *by how much the slope increases* at that knot — between β_k and β_{k+1} the slope of f is $\alpha_1 + \cdots + \alpha_k$. So f is a piecewise-linear curve fitted through the staircase of (mapped value, position) pairs, and it stays monotone — never turning downward, which shard prediction requires — exactly when the knots are sorted ($\beta_1 < \beta_2 < \dots$) and every partial sum of α is non-negative. Training minimises the squared error $\sum_i (f(x_i) - i)^2$ subject to those two constraints.

The optimiser exploits a convenient split between the two parameter sets. With the knots β held fixed, the best α is an ordinary linear least-squares problem with a closed-form answer (one $\sigma \times \sigma$ linear system). The knots themselves have no closed form and are

improved iteratively. The reference trainer therefore alternates, per model: solve for α exactly, take a Newton-style step in β, try ten step lengths and keep the best one that respects the constraints, repeat up to 200 times — in a Python loop over the 64 models, one after another.

4.2.2 One optimisation problem instead of 64

The obvious GPU port — keep the per-model loop, run each model's small linear algebra on the device — is a trap. It launches on the order of 22 kernels per iteration $\times$ 200 iterations $\times$ 64 models $\approx 2.8 \times 10^5$ kernels per training run, each operating on kilobytes; GPU time would be pure launch latency. Our trainer instead treats the 64 models as a single problem.

Each column owns a contiguous slice of the sorted mappings, of length $n_i \approx N/64$ but uneven. The slices are padded to the longest length $n_{\max}$ and stacked into one three-dimensional block of shape (64, $n_{\max}$, σ) once the ReLU terms are expanded. Padded slots are filled with the value -10^{18}, and that choice does the masking by itself: $\max(0, -10^{18} - \beta_j) = 0$ for any sensible knot, so a padded slot contributes exactly zero to predictions, to the loss and to every gradient. No per-slot branching is needed inside the arithmetic; a boolean mask is kept only for the few places where the sentinel would corrupt a reduction (per-column minima, loss sums).

One iteration for all models simultaneously then consists of half a dozen tensor operations: build the ramp activations $A_{m,i,j} = \max(0, x_i^{(m)} - \beta_j^{(m)})$ with one broadcasted subtract-and-clip; form all 64 normal-equation matrices with one `einsum`; solve all 64 systems for α with one batched `cp.linalg.solve` call; assemble gradients and Gauss–Newton Hessians in β from the same ingredients; one more batched solve for the Newton step; then the line search below. The number of kernel launches per iteration is constant — it no longer grows with the number of models, which is the entire point.

4.2.3 Vectorised line search and best-so-far tracking

The reference optimiser's ten-candidate line search is reproduced exactly, but across all models at once: for each of the ten learning rates, candidate β vectors are formed and sorted, α re-solved (another batched solve), validity checked (monotonicity of β, non-negative prefix sums of α), and losses evaluated; an `argmin` along the learning-rate axis then picks each model's best candidate independently. A model whose ten candidates all fail keeps its parameters and is marked inactive via a boolean `active` mask — the batched replacement for the CPU trainer's early exit. Because a model can improve at iteration k and diverge at k+5, the trainer also tracks each model's best-so-far $(\alpha, \beta, \text{loss})$ and returns those, not the final iterate. "Run everything, mask the stragglers" is standard

GPU practice; the bookkeeping that keeps monotonicity validity and best-so-far weights consistent per model is where the implementation effort went.

4.2.4 Three departures from the reference optimiser

Batching forced three behavioural simplifications, all documented in the code and revisited in the threats discussion:

1. *Ridge regularisation replaces condition checks.* The CPU trainer guards every $\sigma \times \sigma$ solve with an explicit condition-number estimate (an SVD per model per iteration) and bails out when ill-conditioned. Per-model bail-outs break batching, so we instead add $10^{-10}I$ to every normal-equation matrix, making the batched solve unconditionally well-posed. The perturbation is roughly eight orders of magnitude below the diagonal scale of the matrices involved.

2. *No monotone re-projection fallback.* The CPU trainer occasionally repairs an invalid α by re-projecting onto the monotone cone. The batched trainer simply rejects the candidate (the model keeps its previous valid parameters). The fallback fires rarely, and its absence is covered by the best-so-far tracking.

3. *Fixed iteration count with masking.* All 200 iterations run even if most models converged early; converged models are frozen by the mask. This wastes some arithmetic but keeps every kernel shape static.

These changes mean the two trainers are not bit-identical. The paired test (Section 4.8) trains both on the same eight-column problem and compares final losses model by model: ratios fall between 0.98× and 1.01× with an overall ratio of 1.00× — the batched trainer is occasionally marginally *better*, since running all 200 iterations can pass the point where the CPU trainer gave up. We did not attempt bit-level validation at production scale.

One engineering note: the largest tensors in the Newton step are the $M \times n_{\max} \times \sigma$ activation and sign matrices, each about 4 GB in fp64 at $N{=}10^7$. The sign matrix's entries are $\{0, 1\}$, so we store it in fp32, halving that allocation at no accuracy cost, and free it explicitly before the line search allocates its own buffers. Even so, training is the pipeline's memory peak: the measured device-pool high-water mark is 2.2 GB at 10^6 and 21.5 GB at 10^7, which makes 24 GB-class cards the practical floor for training at the largest scale we test.

4.3 Batched shard prediction

The query side's hot path — the code where the bulk of every query's execution time concentrates — is `predict_shard_ids`: a `searchsorted` locates each query's column,

a gather pulls that column's (α, β) rows, and one broadcasted ReLU-and-dot-product evaluates Equation (3.2) for all Q queries as a $(Q \times \sigma)$ operation, followed by the divide, clip and offset arithmetic. Model parameters live on the GPU permanently; per call, only the Q mapped values cross PCIe in and Q shard identifiers cross back. Everything downstream — range queries, kNN, inserts — calls an internal variant of the same kernel that takes device-resident inputs and skips the transfers entirely.

4.4 Range queries as a flat candidate index

On the CPU, the three-step recipe of Section 3.1 is a loop: for each query, look up its candidate slice in the flat layout, compare those points against its box, count the survivors. The sizes involved make the loop's shape ill-suited to a GPU: each query owns a *different number* of candidates, so "one thread block per query" means wildly unbalanced work, and "one tensor of all queries' candidates" does not exist — the data is ragged. The GPU path dissolves the raggedness by giving up on per-query structure entirely: build *one flat list of (query, candidate) pairs across all queries* — as if every query wrote each of its candidate row numbers on a card, labelled the card with its own query id, and all cards were stacked into a single deck — then process the whole deck with elementwise operations.

1. Map both corners of every box; predict corner shards with the in-GPU shard predictor; clip to the valid shard range.

2. Convert each query's shard interval into a run of rows of `all_points`, using the flat layout's directory arrays. The run begins where the query's first candidate shard begins — call that row a — and stops where its last candidate shard ends — row b. Shards are stored consecutively, so the candidates are exactly rows $a, a{+}1, \ldots, b{-}1$, and the candidate count is the single subtraction $c = b - a$. Computing a, b and c for all queries at once costs two gathers and one vectorised subtraction.

3. Build the deck — every card's query label and row number — with no loop. To follow the lines, keep a miniature in mind: three queries with candidate counts $(2, 0, 3)$, whose runs begin at rows $(100, 500, 700)$.

```
q_ids  = repeat(arange(n_q), cand_counts)  # which query owns each
    card
ends   = cumsum(cand_counts)               # deck position where
   each block ends
starts = concat([[0], ends[:-1]])          # ... and where each
   block begins
local  = arange(total) - starts[q_ids]     # each card's offset
   inside its block
rows   = run_start[q_ids] + local          # the all_points row
   each card names
```

Line one stamps each query's id once per candidate: $(0, 0, 2, 2, 2)$ — five cards, and the empty query vanishes automatically. Line two takes running totals of the counts, $(2, 2, 5)$: the deck position where each query's block of cards ends. Line three shifts those right, $(0, 2, 2)$: where each block begins. Line four gives every card its position *within its own block*: its absolute deck position (`arange` yields $0, 1, 2, 3, 4$) minus its block's beginning, looked up per card by the gather `starts[q_ids]` $= (0, 0, 2, 2, 2)$ — result $(0, 1, 0, 1, 2)$, read as "I am my query's first card, second card, ...". Line five turns that into an absolute row of `all_points`: the row where the card's query's run begins (another gather, $(100, 100, 700, 700, 700)$) plus the card's offset — final answer $(100, 101, 700, 701, 702)$. Each line is one parallel primitive applied to whole arrays; nothing branches, nothing loops.

4. Gather those rows, evaluate the box predicate for all cards as one broadcasted comparison against each card's own query bounds, and accumulate the survivors per query with a scatter-add (implemented as `bincount` over the matching cards' query labels).

Total work is linear in the number of candidates, fully parallel, and contains no per-query logic at all. One practical complication: at $N{=}10^7$ with wide shard intervals, the flat candidate gather can reach many gigabytes. The implementation therefore processes queries in chunks sized to a fixed candidate budget ($\sim 10^9$ bytes of gathered coordinates), which bounds peak memory at a small constant number of buffers while preserving the no-per-query-logic property within each chunk.

Two CuPy portability quirks are worth recording for reproducibility. `cp.repeat` refuses a device array as its `repeats` argument (NumPy accepts one), so the `repeat` above is implemented as a `searchsorted` over the cumulative counts; and `cupyx.scatter_add` rejects `int64` operands, so the scatter-add is a `bincount` plus addition. Neither workaround changes the asymptotics; both cost an extra kernel launch.

4.5 kNN: lattice inference, box growth, ragged top-k

The kNN path stacks three sub-problems, each of which needed its own GPU reshaping.

Radius prediction. The lattice regression is, in essence, a smooth lookup table: an 11×11 grid of anchor points spans the domain, each anchor holds a learned value ("the typical kNN radius near here"), and a query's prediction is the inverse-distance-weighted average of its 2^d surrounding anchors. Training assigns the anchor values by fitting a sample of points whose true radii are computed by brute force, under a Laplacian smoothness penalty that

keeps neighbouring anchors consistent — a small sparse linear system solved once, at build time. We keep that solve on the CPU (`scipy.sparse.spsolve`); accelerating a one-time 121×121 sparse solve is not worth GPU sparse machinery. Inference is another matter: the reference implementation builds a sparse weight matrix per query batch, but each query in fact touches exactly 2^d anchors, identified by one `searchsorted` per dimension. GPU inference therefore drops sparsity entirely: gather the 2^d rows of B, weight them by normalised inverse distances, and sum — five dense vectorised lines, batched over all queries.

Iterative box growth. Queries whose predicted radius found fewer than k neighbours double their radius and retry (capped at the domain diagonal). On the GPU this loop runs over *rounds* rather than queries: each round extracts the still-unfinished queries, runs the flat candidate scan of Section 4.4 (materialising candidate identities rather than counts), and marks completed queries in a boolean mask. The typical round count is small (one to three); the worst case is bounded by the radius cap. One property of the stopping rule deserves flagging here: a query finishes as soon as its box holds at least k candidates, even when the box is smaller than the distance to the true k-th neighbour — in which case the returned set is the best within the box, not the true answer. Section 6.5 measures how often this happens and what it means.

Ragged top-k. After filtering, each active query owns a different number of in-box candidates, and CuPy has no batched top-k over ragged sets. The implementation assigns every in-box candidate a *slot index* within its query — stable-sort the candidates by query id, subtract each query's cumulative start (computed by `bincount`/`cumsum`) from the candidate's global position — then writes squared distances into a rectangular (queries $\times$ max-count) tensor padded with $+\infty$, sorts along the row axis, and keeps the first k columns. The trick costs one extra sort but turns an inherently ragged selection into two dense primitives.

The CPU kNN path mirrors the same algorithm (lattice radius, box query, exact distances, top-k) with NumPy and per-query Python iteration — the comparison in Section 6.5 is therefore algorithm-for-algorithm, backend-for-backend.

4.6 Dynamic operations as sort problems

The reference LISA inserts one point at a time: predict its shard, walk to the shard's last page, split if full, mutate page lists. As a data structure this is sensible; on a GPU it is unworkable, and reproducing it faithfully would benchmark Python overhead, not hardware. We instead reframed both operations as batch transformations of the flat layout, on *both* backends, so the two sides keep running the same algorithm:

Insert. Predict shard ids for the incoming batch (the same model evaluation a query corner uses); concatenate existing and new points; stable-sort the combined array by shard id; rebuild the per-shard offsets with a `bincount` and `cumsum`. After these three steps the flat layout is exactly what page generation would have produced for the enlarged dataset under the same models. Stability preserves intra-shard order, so existing data never moves relative to itself.

Delete. Pack each row's d coordinates into a single scalar sort key (a scaled sum — collisions are astronomically unlikely on continuous data, and every candidate match is verified against the full row before removal); locate each target's position in the sorted key array with `searchsorted`; mask matched rows out and recompact; rebuild offsets as above.

Both operations are $O((N+B)\log(N+B))$ for batch size B — worse asymptotically than the reference's amortised per-point insertion, but embarrassingly parallel, and the right trade wherever updates arrive in batches. This framing deliberately gives up LISA's per-point update machinery (page splits, shard re-balancing); after enough insertions the shard size distribution will degrade and a periodic re-train/re-layout would be needed. We size the claim accordingly: this is batched update *throughput*, not a full dynamic-index design.

4.7 The monotonic MLP local model

The piecewise-linear model (3.2) is LISA's only learned component, and a natural question is whether a small neural network would fit columns better. The constraint is monotonicity — shard prediction breaks without it. We use the classical device for enforcing it, which goes back to Sill's monotonic networks [16] (where weight positivity is likewise obtained by exponentiating unconstrained parameters) and recurs in the lattice line of work [15]: a $1 \to H \to H \to 1$ MLP whose weights are parameterised as $W = \exp(W_{\text{raw}})$ — strictly positive by construction — with ReLU activations. A composition of non-decreasing functions is non-decreasing, so monotonicity holds for any parameter values; biases stay unconstrained, since they do not affect it. There is no clipping or projection anywhere in training.

All M column models train simultaneously: weights are stacked as $(M, \text{in}, \text{out})$ tensors, every layer is one batched matrix multiply (PyTorch `matmul` broadcasts over the leading axis), the loss is a per-model masked mean of squared rank errors, and one Adam optimiser updates everything for 2000 steps under a cosine learning-rate decay. Inputs and targets are normalised per column to $[0, 1]$ — without this, columns with different mapped-value ranges train at wildly different effective rates. Initialisation also needed a small departure from custom: positive-weight networks have no sign cancellation, so variance-preserving schemes like He initialisation do not apply, and we instead centre $\exp(W_{\text{raw}})$ around

1/fan in to keep layer output scales stable.

Two scoping decisions are important for reading the results. First, the MLP is a *side-by-side comparison only*: it is not wired into `predict_shard_ids`, and no query in this thesis is answered by an MLP. Second, the comparison metric is final training loss against the batched piecewise trainer on identical inputs. As Section 6.7 shows, the MLP trains somewhat faster and fits far worse — between 2.4× and five orders of magnitude worse depending on dataset and scale, with a single interesting exception on heavily skewed real data — which is why integration was not pursued.

4.8 Correctness testing without a GPU

Every stage has a paired correctness test that runs the *GPU code path* under the NumPy backend, so the logic is testable on any machine:

- batched trainer vs. sequential CPU trainer: per-model final losses within 0.98–1.01× on the paired problem (Section 4.2);
- range query vs. brute force: exact match on counts and contents across randomised boxes;
- kNN vs. brute force: exact distance agreement on 29 of 30 randomised queries, the residual case being a radius that saturated at its cap before reaching k candidates — the documented approximate regime of LISA's kNN;
- insert/delete: point-count and findability invariants hold after each operation (inserted points are returned by subsequent whole-domain queries; deleted points are not);
- MLP: monotonicity spot-checks on dense input sweeps, and a sanity bound against a trivial baseline.

On GPU runs, the benchmark additionally cross-checks CPU and GPU results in place: sorted mappings agree to 10^{-6} after the build, shard predictions match exactly, and range-query counts match exactly on every configuration reported in Chapter 6.

5. Experimental Setup

5.1 Hardware and software

Experiments ran on two Linux cloud instances, summarised in Table 5.1. The primary machine pairs an Intel Xeon host with an NVIDIA A100-SXM4 (80 GB, Ampere generation) — a standard datacenter GPU whose characteristics are widely known, which makes the absolute numbers easy to interpret. The secondary machine is the same instance class with an NVIDIA L4, a mid-range card included to show how the results transfer to smaller hardware; it ran the full benchmark at $N \in \{10^5, 10^6\}$. The two instances report the same host CPU model, which makes the device-to-device comparison in Section 6.8 unusually clean. Both ran Python 3.12.13, NumPy 2.0.2, CuPy 14.0.1 and CUDA 12.8.

	Primary	Secondary
GPU	NVIDIA A100-SXM4	NVIDIA L4
GPU memory	79 GB	22 GB
GPU generation	Ampere	Ada Lovelace
CPU	Intel Xeon @ 2.20 GHz	Intel Xeon @ 2.20 GHz
CUDA / driver	12.8 / 580.82	12.8 / 580.82

Table 5.1: The two evaluation machines. Memory figures are as reported by the CUDA runtime.

The GPU brute-force kNN baseline is exact: dense pairwise L2 distances plus top-k selection, implemented with PyTorch (`cdist` followed by `topk`) and chunked over queries so the distance matrix stays within device memory at $N{=}10^7$.

5.2 Datasets

Three data distributions, each at $N \in \{10^5, 10^6, 10^7\}$:

Uniform. Points drawn uniformly from $[0, 10^4]^2$ — LISA's friendliest case, since equi-depth cells then have near-identical geometry and the rank function within each column is nearly linear.

Skewed (synthetic). A mixture of twenty Gaussian clusters with Dirichlet-distributed weights and varying spreads over an 80% mass, plus 20% uniform background — a stand-in for the power-law density of urban geographic data.

GeoNames (real). The GeoNames `allCountries` gazetteer [18], roughly 12–13 million named geographic features worldwide. We use longitude/latitude as coordinates, drop

incomplete rows, subsample without replacement to the requested N (bootstrap with jitter would apply below 10^5, but all our sizes fit), and rescale.

All datasets, including GeoNames, are affinely rescaled per axis into $[0, 10^4]^2$. Coordinates and distances in this thesis are therefore *unitless*: a range box of half-width 50 covers 0.5% of each axis of the rescaled domain, not any particular number of kilometres, and match counts depend on the rescaled density only. This does not affect CPU-to-GPU or index-to-index comparisons, which all see identical data, but absolute radii and match counts should not be read geographically.

Data generation is seeded (seed 42); range-query and kNN workloads use seeds 1 and 99 respectively. Re-running the released harness with these seeds reproduces the exact inputs of every experiment.

5.3 Workloads

Shard prediction. $Q = 10^4$ mapped values drawn uniformly over the mapped domain; the measured operation is the batched `predict_shard_ids` alone. Throughput is reported as lookups per second.

Range queries. $Q = 1000$ axis-aligned boxes with centres uniform over the domain and per-axis half-widths drawn from $[0.25\%, 0.75\%]$ of the axis span (mean 0.5%). Box size directly determines result sizes — means of about 10, 99 and 986 matches per query at the three uniform scales — and therefore filter work; larger boxes would shift the balance further toward the scan-and-filter stage. The measured operation is the full pipeline: corner mapping, corner shard prediction, candidate scan, exact filter, per-query counts.

kNN. $Q = 500$ query points, $k = 10$. The lattice radius model is trained once per dataset size on 300 sampled points whose true kNN radii are computed by (chunked) brute force; this setup cost is reported separately since it is one-time, and it is dominated by the radius sampling, not the lattice solve (Section 6.5).

Dynamic operations. Batches of 10^3, 10^4 and 10^5 new uniform points inserted into the built index, then deleted again; reported as wall time per batch and as speedup.

5.4 Measurement protocol

Every timed figure is the mean of 3 repetitions (standard deviations are retained in the released CSV files). GPU timing brackets every region with CUDA stream synchronisation; host-to-device and device-to-host transfer times are measured separately from compute and reported where relevant, so "GPU total" always means *including* per-call transfers. The

CUDA context is initialised by a warm-up operation before any timed region. CPU model training repeats fewer times ($\max(1, \text{reps}/2)$) because a single repetition takes minutes.

Two protocol notes need stating up front; we believe stating them plainly is preferable to letting them surface from the data tables.

Model training and weight provenance. The sequential CPU trainer is run only up to $N = 10^6$; at 10^7 a single repetition extrapolates to well over an hour on this host (observed: 513 s at 10^6), so the harness skips it and GPU training time at 10^7 is reported as an absolute number without a CPU speedup factor. The query stages of both backends always share identical model weights: the sequential trainer's at 10^5 and 10^6, the batched GPU trainer's at 10^7. The weight source is recorded per row in the released data.

First-run JIT compilation at N=10^5. CuPy compiles each kernel specialisation on first use and caches it on disk. The uniform dataset was benchmarked first on a fresh instance, so its N=10^5 GPU figures — the first invocations of each kernel — absorb a few hundred milliseconds of one-time compilation in one of three repetitions, visible as standard deviations exceeding the means in the released data. The skewed and GeoNames runs, executed afterwards against a warm kernel cache, show the steady-state behaviour at 10^5 (e.g. range query: 7.3 ms versus the contaminated 549 ms). Where a uniform-10^5 number is affected we either flag it or quote the steady-state runs instead; a cleaner protocol (discarding a warm-up repetition per kernel) is noted in Section 5.6.

5.5 Baselines

Four classical baselines run against the *identical* point sets and query sets (same seeds, same flat layout source):

- **R-tree** (`libspatialindex` via the `rtree` package), STR bulk-loaded, for range queries. Queries execute one at a time through the package's Python binding — there is no batch interface — so its measured times include per-query Python overhead; we discuss the implications alongside the results.
- **Quad-tree** (`pyqtree`, pure Python), for range queries; the same per-query caveat applies, more strongly.
- **cKDTree** (SciPy, C implementation) for kNN, queried through its native batch interface — effectively the strongest practical CPU baseline available off the shelf.
- **GPU brute force** for kNN: exact pairwise distances plus top-k, as described above. Unlike every other method here, it guarantees exact answers at any scale, which matters for the accuracy discussion in Section 6.5.

The R-tree, quad-tree and `cKDTree` numbers come from a standalone baseline harness run in a separate process on the same instance, against the identical point sets and query sets; the main benchmark's own in-run R-tree measurement agrees with the standalone one to within about 3%, which is the kind of cross-check that costs nothing and catches harness bugs.

The asymmetry in implementation languages cuts both ways and is unavoidable in a study of this size: LISA-CPU is NumPy-vectorised Python, the R-tree core is C++ behind a per-query Python binding, the quad-tree is pure Python, and `cKDTree` is C with a batch API. We therefore treat baseline comparisons as ecosystem-level statements — "what a practitioner would get from the standard Python stack" — rather than as language-neutral algorithmic verdicts, and we say so again where it matters in Chapter 6.

5.6 Threats to validity

For convenience, the main caveats in one place. (i) The CPU LISA is the public Python/NumPy implementation; the native implementation behind the original paper's experiments was never released. Speedups versus a native CPU LISA would be smaller; the *trends* across N, which depend on algorithmic structure rather than constant factors, are the transferable result. (ii) Uniform-10^5 GPU numbers include one-time JIT compilation, as discussed above. (iii) The CPU numbers are effectively single-core: NumPy multi-threads only its BLAS-backed routines, and the hot loops here (sorts, searchsorted, elementwise filtering) are not among them. This frames the results rather than undermining them — even a perfectly parallel CPU implementation on a 32-core machine could close a 3–5× gap but not a 60–570× one — and the affected claims are pointed out where they appear.

6. Results

All numbers in this chapter are from the primary (A100) machine unless stated otherwise; Section 6.8 compares against the L4. Tables report means over three repetitions. The complete CSV outputs, including standard deviations and per-row weight provenance, accompany the thesis.

6.1 Build phase: partitioning and sorting

The build phase behaves exactly as its restructuring (Section 4.1) predicts: it is sort-bound, and large GPU sorts are extremely fast. Table 6.1 and Figure 6.1 show partition reaching 573× at $N{=}10^7$ — a single 10-million-element sort simply vanishes on this hardware (5.2 ms) — while mapping+sort, which interleaves gathers and elementwise arithmetic with its global sort, settles around 20×. PCIe transfers (0.17 s round trip at 10^7) cost over thirty times the partition kernel itself, foreshadowing a recurring theme: at these sizes the GPU's problem is feeding it, not arithmetic. Below $N \approx 10^6$ the CPU is already so fast (tens of milliseconds) that GPU execution is pointless overhead even in steady state — mapping+sort at 10^5 on a warm cache is roughly at parity.

6.2 Model training

Training (Table 6.2, Figure 6.2) delivers the largest end-to-end build win, and its behaviour is worth examining closely. Three observations follow.

First, the batched trainer wins everywhere — 5× at 10^5, 13–27× at 10^6 — and the win grows with N as the fixed kernel schedule amortises over larger column tensors. The A100 helps here in a specific way: the Newton step is built from fp64 solves and `einsum`s, and this card executes double precision at half its single-precision rate, an unusually generous ratio. The remaining ceiling is structural: a Newton-step solve plus one α-solve per line-search candidate comes to roughly a dozen batched $\sigma{\times}\sigma$ solves per iteration, so per-iteration kernel count, not arithmetic, is what limits the small-N end.

Second, the speedup is data-dependent, and the pattern is informative. The CPU trainer exits a model's loop as soon as its line search stops improving, and on hard-to-fit data (GeoNames columns contain near-duplicate mapped values and extreme density spikes) most models give up early: the sequential trainer's GeoNames time at 10^6 (193 s) is well under half its uniform time (513 s). The batched trainer cannot exploit per-model early exit in the same way — a batch advances at the pace of its slowest member — so its advantage compresses from 26.5× on uniform data to 12.9× on GeoNames. Batching

N	stage	CPU (s)	GPU (s)	speedup
10^5	partition	0.0150	0.1759†	—
10^5	mapping+sort	0.0111	0.3443†	—
10^6	partition	0.2122	0.0029	74.2×
10^6	mapping+sort	0.1308	0.0077	16.9×
10^7	partition	2.978	0.0052	572.9×
10^7	mapping+sort	1.850	0.0828	22.4×

Table 6.1: Build-stage times, uniform data. GPU columns are kernel time; transfers add 0.17 s round-trip at $N{=}10^7$. †Contaminated by one-time kernel compilation (Section 5); the warm-cache GeoNames run measures 0.0032 s and 0.0157 s for the same stages at 10^5, i.e. a 4.6× speedup on partition and rough parity on mapping+sort.

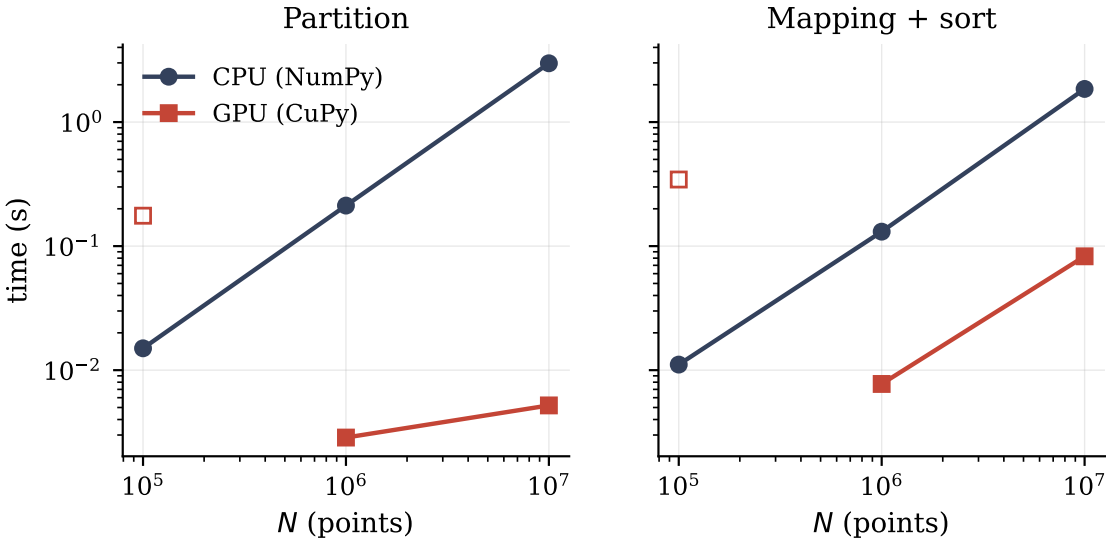

Figure 6.1: Build-stage scaling on uniform data (log–log). Open marker: the $N{=}10^5$ GPU point inflated by one-time kernel compilation.

trades adaptivity for uniformity; the trade is still clearly worth it here, but a two-phase schedule (drop converged models from the batch and re-pad periodically) would claw back more and is noted as future work.

Third, the batched trainer is what makes $N{=}10^7$ *feasible* at all: 146–166 s on the GPU versus an extrapolated hour-plus per repetition on the CPU. Removing the pipeline's dominant cost at scale was the stated goal of this stage, and it is met, with the caveat that the comparison at 10^7 is against an absent baseline.

Fit quality matches the sequential trainer on the paired test problem (per-model loss ratios 0.98–1.01×, Section 4.8). At benchmark scale we did not bit-compare the two trainers' weights; the documented optimiser simplifications make some divergence expected, and quantifying it at $N{=}10^6$ is listed in Chapter 7.

dataset	N	CPU sequential (s)	GPU batched (s)	speedup
uniform	10^5	64.60	12.20	5.3×
uniform	10^6	513.08	19.34	26.5×
uniform	10^7	—	166.1	—
skewed	10^5	62.13	5.35	11.6×
skewed	10^6	380.97	15.27	25.0×
skewed	10^7	—	147.3	—
GeoNames	10^5	32.70	5.90	5.5×
GeoNames	10^6	192.65	14.94	12.9×
GeoNames	10^7	—	145.7	—

Table 6.2: Training all 64 piecewise-linear models (σ=50, up to 200 Newton iterations). CPU training is skipped at N=10^7 (a single repetition extrapolates to well over an hour); GPU times there are absolute, with no speedup factor claimable.

dataset	N	CPU (ms)	GPU tot. (ms)	GPU cmp. (ms)	$\text{speedup}_{\text{tot}}$	$\text{speedup}_{\text{cmp}}$
uniform	10^6	4.77	1.23	1.03	3.9×	4.6×
uniform	10^7	5.24	1.43	1.15	3.7×	4.6×
skewed	10^5	11.51	2.65	2.44	4.3×	4.7×
skewed	10^6	4.62	1.19	1.00	3.9×	4.6×
skewed	10^7	5.15	1.22	1.03	4.2×	5.0×
GeoNames	10^5	6.14	2.54	2.35	2.4×	2.6×
GeoNames	10^6	9.88	3.24	3.04	3.0×	3.2×
GeoNames	10^7	5.18	1.23	1.03	4.2×	5.0×

Table 6.3: `predict_shard_ids` for $Q = 10^4$ lookups (the uniform 10^5 row is omitted as compilation-contaminated). "Total" includes per-call PCIe transfers.

6.3 Shard prediction

The learned lookup itself (Table 6.3, Figure 6.3) shows two fast implementations side by side. The CPU path is a well-vectorised NumPy expression processing 10^4 lookups in roughly 5–10 ms — one to two million lookups per second on one core — and the GPU multiplies that by 3–4×, to about eight million lookups per second including transfers. Note the latency floor: GPU compute time sits at 1.0–1.2 ms across two orders of magnitude of N (the $(Q \times \sigma)$ tensor does not depend on N, and at this size the kernel sequence is launch-bound, not arithmetic-bound). Larger query batches would raise the achievable ceiling; we did not evaluate beyond $Q = 10^4$. The GeoNames rows carry visibly noisier CPU times (the released data includes the standard deviations); the GPU side is steady throughout.

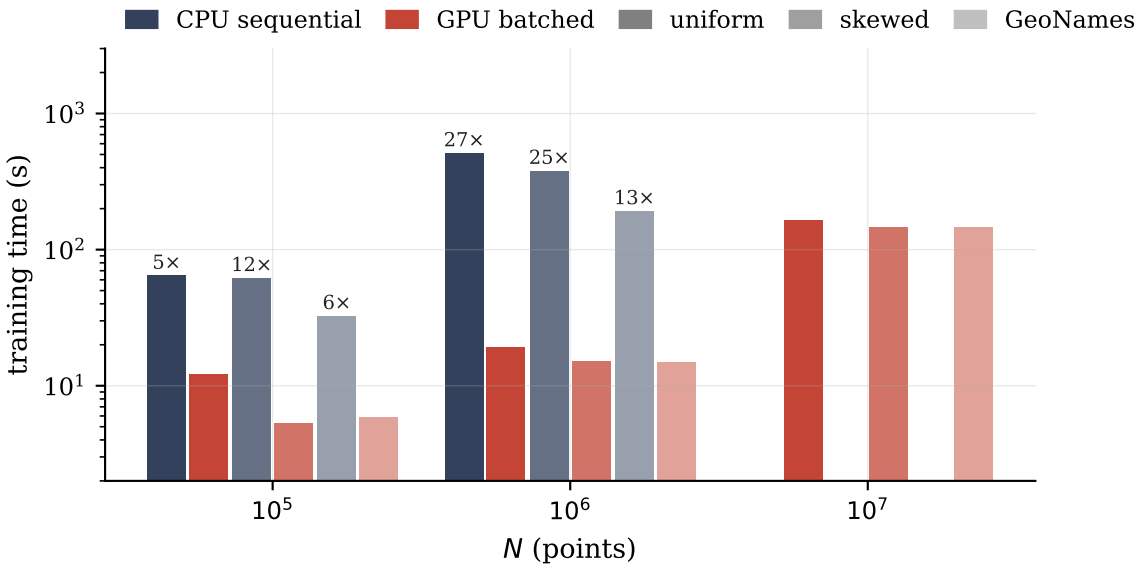

Figure 6.2: Training time by dataset and scale (log axis). Annotations give the GPU speedup where a CPU measurement exists. At $N{=}10^7$ only the batched GPU trainer is practical.

6.4 Range queries

Range queries are the headline result (Tables 6.4–6.5, Figure 6.4). Three layers of comparison:

GPU vs. CPU LISA. The flat-candidate formulation delivers 60–63× at 10^6 and 95–100× at 10^7, with near-identical numbers across all three data distributions — evidence that the formulation has no distribution-sensitive control flow left in it. Per-query cost on the GPU is 5.9 μs at 10^6 and 35 μs at 10^7: candidate volume grows linearly with N at a fixed box fraction, but the GPU absorbs a tenfold candidate increase with only a sixfold time increase, because part of each call is fixed overhead — hence the speedup *rising* with scale. Even at 10^5 — where the contaminated uniform row is misleading — the steady-state runs show a useful 6.5–7×.

GPU LISA vs. classical CPU structures. Against the bulk-loaded R-tree, GPU LISA answers the same thousand boxes 11–17× faster, and the margin holds steady from 10^6 to 10^7; against the quad-tree it is 15–46×. One caveat runs in the R-tree's favour: it pays per-query Python binding overhead through the `rtree` package, so a batch-capable native R-tree would narrow these margins somewhat.

CPU LISA vs. everything. The NumPy implementation itself, for completeness, loses to the R-tree at every scale (0.11–0.66×): it is an algorithmic reference point, not a competitive CPU system, which is why the headline ratios above should be read as GPU-versus-NumPy (Section 5.6, item i).

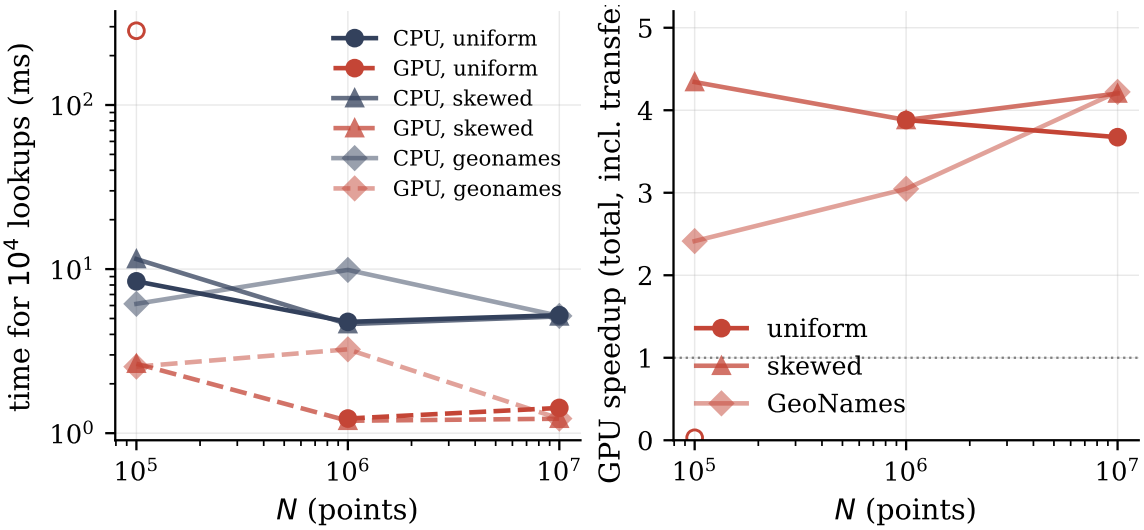

Figure 6.3: Left: batched shard-prediction time for 10^4 lookups (log–log; open marker = compilation-contaminated point). Right: total GPU speedup including transfers. The GPU settles between 2.4× and 4.3× wherever steady-state measurement exists.

dataset	N	matches/q	CPU (s)	GPU (s)	speedup	GPU μs/query
uniform	10^5	9.8	0.0518	0.5487†	—	—
uniform	10^6	98.7	0.3727	0.0059	62.7×	5.9
uniform	10^7	986.3	3.499	0.0350	99.9×	35.0
skewed	10^5	12.6	0.0484	0.0074	6.5×	7.4
skewed	10^6	124.9	0.3444	0.0058	59.6×	5.8
skewed	10^7	1253.4	3.171	0.0332	95.5×	33.2
GeoNames	10^5	7.6	0.0492	0.0073	6.8×	7.3
GeoNames	10^6	76.7	0.3569	0.0059	60.4×	5.9
GeoNames	10^7	925.0	3.639	0.0367	99.2×	36.7

Table 6.4: Range queries: 1,000 boxes (half-width ≈ 0.5% of span), full pipeline, GPU times including transfers. †Compilation-contaminated (steady state at this size is ≈ 7 ms, as the other two datasets show).

Build-cost context belongs in any fair reading: the R-tree bulk-loads 10^7 points in 46 s and the quad-tree needs 125 s, while LISA's full build at that scale is dominated by 166 s of GPU training — the measured partition, sort and transfer stages add under 0.3 s, and only the one-time CPU page-generation step (not separately instrumented) sits outside these figures. Batched training has pulled LISA's construction cost into the same band as the classical structures it competes with, which was not true of the sequential trainer. LISA's storage profile is the mirror image: the learned component is ∼ 52 KB of parameters plus 1.6 MB of shard offsets, while tree structures carry per-point entries. The trade is build time and update machinery for query throughput and memory — a trade that favours LISA precisely in read-heavy, batch-query settings.

		query, 1,000 boxes (s)		build (s)		LISA-GPU
dataset	N	R-tree	quad-tree	R-tree	quad-tree	vs. R-tree
uniform	10^6	0.0719	0.1325	4.38	11.7	12.1×
uniform	10^7	0.4345	1.2094	46.4	125.1	12.4×
skewed	10^6	0.0871	0.1631	4.37	12.4	15.1×
skewed	10^7	0.5506	1.5422	46.5	133.4	16.6×
GeoNames	10^6	0.0625	0.0880	4.59	12.7	10.6×
GeoNames	10^7	0.4094	1.0504	48.5	141.6	11.2×

Table 6.5: Classical range-query baselines on identical data and queries. The last column compares GPU LISA's batch time against the R-tree's. Both baselines answer queries one at a time through Python bindings; see the fairness discussion in Section 5.5.

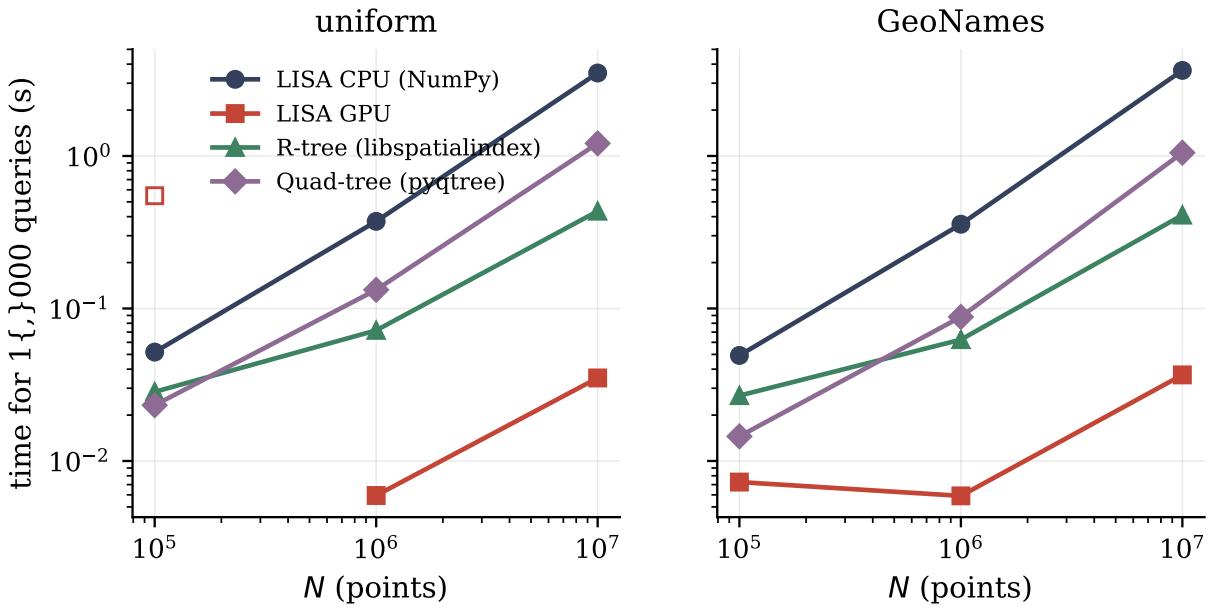

Figure 6.4: Range-query batch time vs. N, log–log, on uniform (left) and GeoNames (right). Open marker: compilation-contaminated point. GPU LISA is fastest at every scale on both distributions, with a steady margin over the R-tree.

6.5 kNN queries

The kNN results need reading on two axes at once: speed and correctness. Neither favours the design.

Speed. The GPU code does its local job (Table 6.6, Figure 6.5): algorithm-for-algorithm it beats CPU LISA by 3.8–6.8×, with the batched lattice inference and ragged top-k behaving as designed, and at 10^7 it even undercuts exact GPU brute force on uniform and skewed data (67 vs. 168 ms). That last comparison is less favourable than it appears, however — brute force returns exact answers and, as shown below, LISA here does not — and the comparison that matters is unambiguous: a single-core `cKDTree` answers the same 500 queries in 1.8–2.9 ms, between 12× and 350× faster than GPU LISA. Skew remains

dataset	N	LISA CPU	LISA GPU	speedup	GPU brute	cKDTree
uniform	10^6	113.9	24.7	4.6×	18.0	1.99
uniform	10^7	361.4	66.8	5.4×	167.7	2.38
skewed	10^6	259.0	67.4	3.8×	17.0	1.82
skewed	10^7	800.9	144.1	5.6×	166.4	2.30
GeoNames	10^6	1221.3	190.6	6.4×	16.5	2.20
GeoNames	10^7	7008.7	1023.8	6.8×	166.1	2.91

Table 6.6: kNN: 500 queries, k=10, times in milliseconds. "GPU brute" is exact pairwise distance + top-k (Section 5.5); `cKDTree` build times are 0.32–0.33 s (10^6) and 4.1–4.4 s (10^7).

expensive: on GeoNames the 300-sample radius model misjudges the wildly varying local density, retry rounds multiply, and CPU LISA's kNN at 10^6 is eleven times its uniform cost (1.22 s vs. 0.11 s); the GPU inherits the same behaviour at a smaller constant.

There is also a setup cost the harness now itemises: building the radius model's training set — brute-forcing true kNN radii for 300 sample points — takes 14.6 s at 10^6 and 147–149 s at 10^7 on the CPU, dwarfing the lattice solve itself ($\sim$ 3 ms). It is a one-time cost, but at 10^7 it rivals the entire model-training stage, and it is an obvious GPU porting target (Chapter 7).

Correctness. The stopping rule flagged in Section 4.5 — accept a box as soon as it holds k candidates — has measurable consequences, which we quantify with a controlled experiment using exact k-d-tree ground truth. On uniform data at 10^6, the true 10-th-neighbour distance at the 500 query points is 18.7 ± 2.7 units, while the lattice predicts 18.2 ± 0.6: the prediction is nearly unbiased *on average* but far smoother than the quantity it predicts, because a 300-sample lattice can only learn the local mean, not the per-query fluctuation. The predicted box then holds about 13 candidates on average — enough to satisfy the $\geq k$ acceptance test — yet for 57% of queries the box half-width is below that query's true 10-th-neighbour distance, so at least one true neighbour lies outside the box and the returned set is only the best *within* it. At 10^7 the picture is the same (5.6 ± 0.3 predicted vs. 5.8 ± 0.9 true; 59% of queries truncated). On clustered data the lattice's smoothing over-predicts radii in dense regions, which restores exactness for many queries at the price of larger boxes — part of why GeoNames is slow.[1]

The conclusion has to be stated as measured: *at d=2 and $N \leq 10^7$, LISA's kNN-via-ranges design is not competitive with classical alternatives on either backend, and the implementation's stopping rule makes its answers approximate for over half the queries on*

[1]The benchmark also records a strict per-distance match rate against the GPU brute-force reference. We do not report it as a recall figure: the reference computes distances in fp32, and at our coordinate magnitudes its rounding error exceeds the 10^{-3} match tolerance for most pairs at 10^6–10^7, which we verified directly. The controlled analysis above, built on exact k-d-tree distances, is the accuracy statement this thesis stands behind.

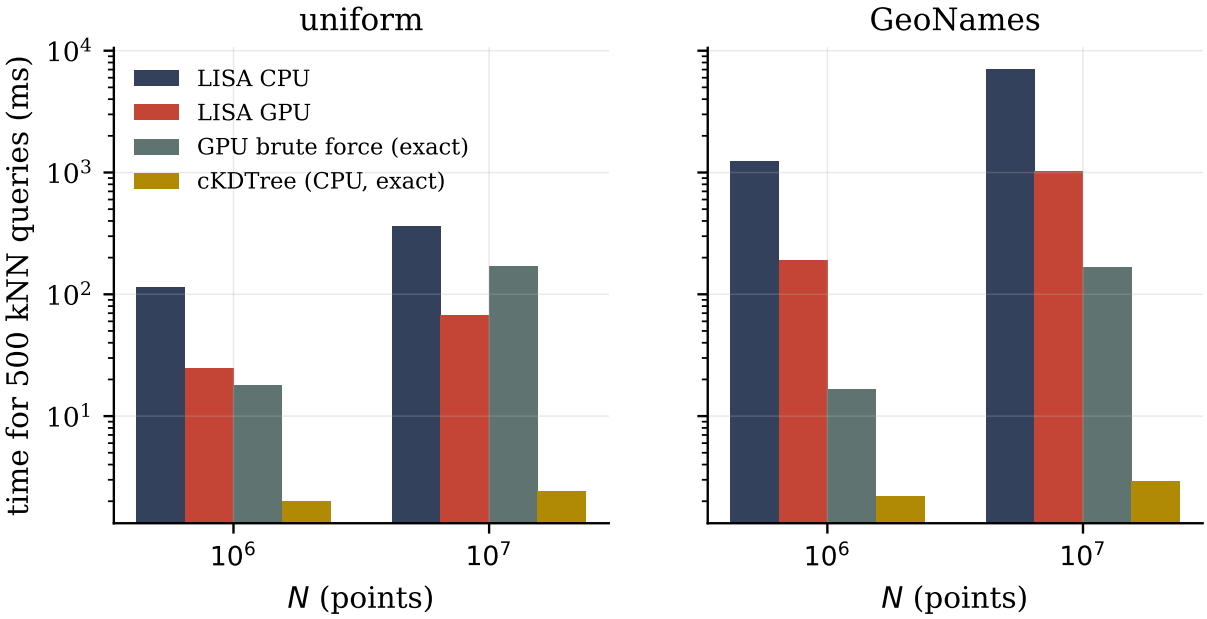

Figure 6.5: kNN comparison (log scale, lower is better). GPU LISA beats CPU LISA by 4–7× and, at 10^7, exact GPU brute force on two of three datasets — but `cKDTree` remains one to two orders of magnitude ahead, and Section 6.5 shows LISA's returned sets are approximate for many queries.

uniform data. The accuracy half has a cheap fix — grow the box until the k-th candidate's distance is no larger than the box half-width, rather than until k candidates exist — at the price of more growth rounds; it is recorded in Chapter 7. The speed half does not condemn learned kNN generally: k-d-tree pruning collapses in higher dimensions where learned and brute-force methods scale more gracefully, and the original LISA paper motivates its design through I/O cost on disk-resident data, not in-memory latency, which is what we measure here.

6.6 Dynamic operations

The sort-and-rebuild framing (Section 4.6) was designed for batch throughput, and that is where it delivers (Table 6.7, Figure 6.6): inserting 10^5 points into a 10^6-point index takes 5.7 ms on the GPU — over 17 million points per second — and into a 10^7-point index 28 ms, still 3.6 million per second against 0.53 s on the CPU. The speedup curve has the expected shape: modest at small batches (1.6× at B=10^3, N=10^5, where re-sorting N+B keys is overkill for a thousand arrivals) and saturating around 19–27× once the batch is large enough to amortise the sort. Per-point insertion is explicitly the wrong workload for this design: the claim is batched throughput, nothing more.

The delete column needs a qualifier of its own. The 108–268× ratios are real measurements but overstate the GPU's advantage, because the CPU deletion path — a lexicographic

		insert			delete		
N	batch	CPU (s)	GPU (s)	speedup	CPU (s)	GPU (s)	speedup
10^5	10^3	0.0049	0.0030	1.6×	0.089	0.818†	—
10^5	10^5	0.1080	0.0041	26.3×	0.329	0.0031	108×
10^6	10^3	0.0403	0.0055	7.4×	0.940	0.0050	189×
10^6	10^5	0.1525	0.0057	26.6×	1.198	0.0059	203×
10^7	10^3	0.4167	0.0284	14.7×	10.19	0.0380	268×
10^7	10^5	0.5295	0.0280	18.9×	10.30	0.0392	262×

Table 6.7: Batched insert/delete, uniform data (selected rows). †First-call compilation (the clean steady-state value at this cell is ≈ 7 ms, as the other datasets show). The delete speedups overstate the GPU's algorithmic merit — see text.

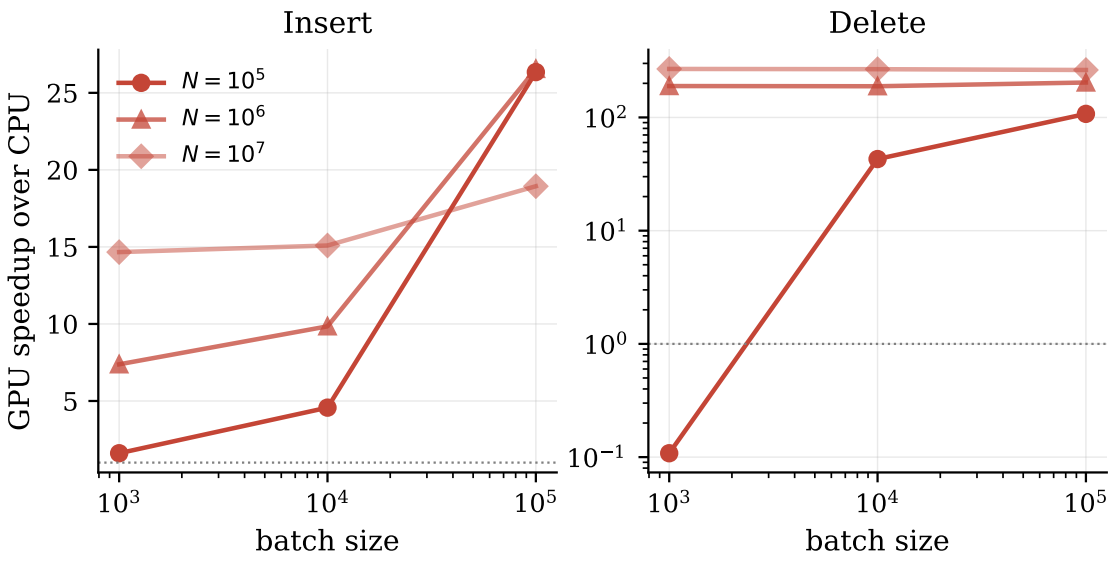

Figure 6.6: Dynamic-operation speedup vs. batch size (uniform data). Insert saturates around 19–27×; the delete panel's much larger ratios (log axis) partly reflect a weak pure-Python CPU baseline.

two-pointer match written in pure Python, costing over ten seconds per batch at 10^7 — is the weakest CPU baseline in this thesis; a vectorised NumPy deletion would land far closer to the insert column's ratios. We therefore rest the dynamic-operations claim on the insert numbers and report the delete ones with that caveat attached.

6.7 The monotonic MLP: exploratory and unconvincing

The monotonic MLP trains in roughly two-thirds to three-quarters of the batched Newton trainer's time — and fits badly (Table 6.8, Figure 6.7). On uniform data its final loss is 72× to 1600× the piecewise model's; on synthetic skewed data the gap explodes to five orders of magnitude. The shape of the failure is consistent with the architecture: a

dataset	N	PWL train (s)	MLP train (s)	loss ratio MLP/PWL
uniform	10^5	12.2	8.5	72×
uniform	10^6	19.3	12.0	311×
uniform	10^7	166.1	114.1	1562×
skewed	10^5	5.3	4.6	1.98×10^4
skewed	10^6	15.3	12.0	1.13×10^5
skewed	10^7	147.3	114.0	1.73×10^5
GeoNames	10^5	5.9	4.4	5816×
GeoNames	10^6	14.9	12.0	2.35×
GeoNames	10^7	145.7	114.0	0.21×

Table 6.8: Monotonic MLP (H=32, 2000 Adam steps) vs. the batched piecewise trainer on identical column data. Ratios above 1 mean the MLP fits worse. The GeoNames-10^7 inversion is discussed in the text.

positive-weight ReLU network is a smooth monotone regressor, and column rank functions are piecewise-linear with sharp density transitions that fifty explicit hinge points capture directly and a 1→32→32→1 smooth monotone map does not.

The one inversion — GeoNames at 10^7, where the MLP's loss is 0.21× the piecewise model's — needs a careful reading: it is not a case of the MLP fitting well but of *both* models fitting catastrophically (absolute losses of 4.7×10^{13} and 2.2×10^{14} respectively against columns of 1.6×10^5 points), with the smoother model degrading more gracefully. An interesting data point about robustness on hard real-world columns; not a usable result.

Accordingly we state the stage's status exactly: *the monotonic MLP is a future-work direction; in its current form it converges to a much worse fit than the σ=50 piecewise-linear model on essentially all evaluated workloads, it is not wired into* `predict_shard_ids`*, and no query in this thesis is served by it.* What survives from the experiment is machinery, not a model: the batched-across-columns training harness and the exp-parameterisation [16] are sound, and Chapter 7 sketches the variants (hinge-aware architectures, per-column capacity, loss reweighting) that would make a second attempt meaningful.

6.8 Across GPUs: A100 vs. L4

Running the same benchmark on an L4 (Table 6.9, Figure 6.8) answers a practical question: how much of the result is the algorithm, and how much is the large GPU? The trend is the same on the smaller card — every stage keeps its character, the build is still sort-bound and fast, the range path is still the standout, and training still beats the host CPU comfortably (3.4× on the L4 itself, 554 s → 163 s). What changes between the cards is exactly what the workload analysis predicts. The stages bound by memory bandwidth and raw compute stretch by 1.8–2.7× on the L4, and training stretches by 8.4× — the Newton step is

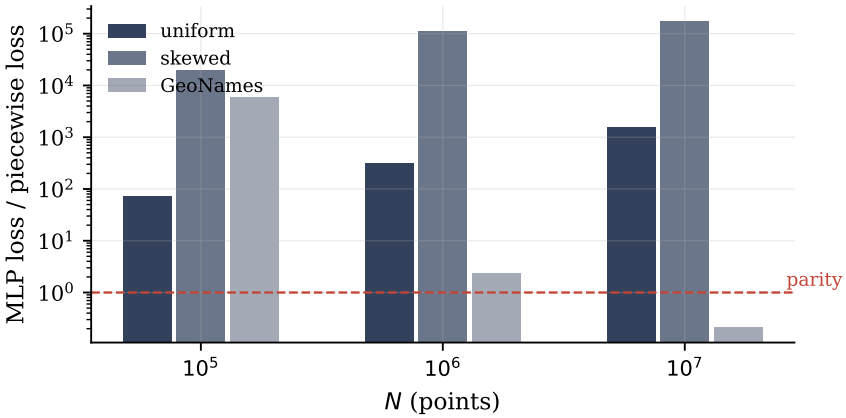

Figure 6.7: MLP-to-piecewise loss ratio (log scale; below the dashed line means the MLP fits better). Only GeoNames at 10^7 crosses parity, and there both models fit poorly.

stage ($N = 10^6$, uniform)	A100	L4	L4 / A100
partition (s)	0.0029	0.0023	0.8×
mapping+sort (s)	0.0077	0.0141	1.8×
model training (s)	19.3	163.2	8.4×
shard prediction, compute (ms)	1.03	0.81	0.8×
range query, compute (ms)	5.5	14.6	2.7×
kNN, compute (ms)	24.2	44.3	1.8×

Table 6.9: GPU-side times at N=10^6 (uniform). The two instances share the same host CPU model, so this is close to a pure device-to-device comparison.

double-precision linear algebra, and the A100 is unusually strong there (fp64 at half its fp32 rate, where the L4 runs fp64 at a small fraction of fp32). The launch-bound stages, by contrast, do not reward the bigger device at all: shard prediction and the 10^6-element partition are marginally *faster* on the L4, because at these sizes the cost is kernel launches, not silicon. Nothing about the approach requires a flagship GPU; larger hardware lowers the throughput-bound costs and leaves the latency floors unchanged.

6.9 When the GPU does not help

Collecting the small-scale behaviour in one place: below $N \approx 10^5$, GPU execution is at best marginally useful and sometimes counterproductive. In steady state at N=10^5 the GPU still wins range queries (6.5–7×) and shard prediction (2.4–4.3×), wins training (5–12×), roughly breaks even on kNN (1.1–3.6× across datasets) and loses mapping+sort and small dynamic batches (1.6× at B=10^3 is not worth a device round-trip). Two fixed costs dominate this regime: kernel-launch floors (every GPU query stage here bottoms out around 1–2 ms regardless of input size) and, in a cold process, one-time kernel compilation measured in hundreds of milliseconds. For an interactive, small-data workload, the CPU implementation — or indeed any classical structure — is the right tool; the GPU pipeline

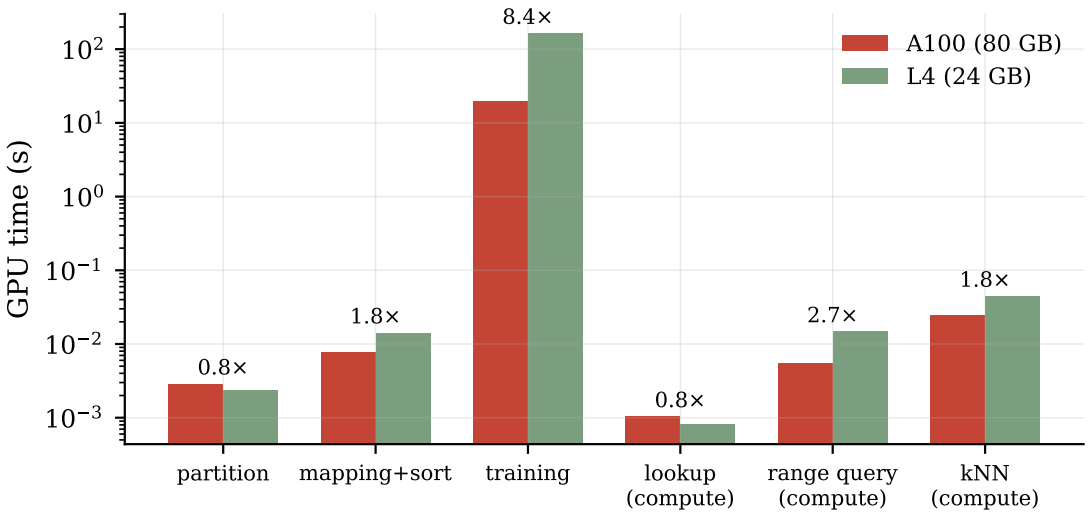

Figure 6.8: Device-side stage times at $N{=}10^6$, A100 vs. L4 (log scale; annotations show the L4/A100 ratio). Throughput-bound stages favour the A100, launch-bound stages do not.

becomes worthwhile from 10^6 points and batch-shaped workloads upward.

7. Limitations and Future Work

Some of what follows repeats caveats already attached to individual results; collecting them, with the corresponding next steps, is the point of this chapter.

The CPU baseline is NumPy. A native (C/C++) LISA would compress every CPU-vs-GPU ratio in this thesis by some constant factor. The build-phase and range-query ratios have enough headroom to survive that; the 3–7× stages (shard prediction, kNN) might not. Re-baselining against a native implementation — the original authors', should it ever be released, or an independent one — is the single most valuable robustness check this work could receive.

The candidate set is wider than it needs to be. A box query scans every point stored between the shard of the box's lower corner and the shard of its upper corner. That interval is guaranteed to contain all the answers, but it also drags in points that merely have mapped values between the two corners without being anywhere near the box — and the number of such points grows linearly with N at a fixed box size, which is why per-query GPU time rises from 5.9 to 35 μs across the tenfold step from 10^6 to 10^7 in Table 6.4. The original paper prunes the interval by checking which grid cells the box actually overlaps and keeping only those cells' stretches of it. Implementing that pruning on the GPU — a batched interval intersection, well suited to the same `repeat/cumsum` machinery — should cut candidate volume by a large factor on small boxes and is the highest-value query-side improvement available.

Training gaps. The batched trainer's three simplifications are measured as benign at test scale but not bit-validated at production scale; a per-model convergence comparison at $N{=}10^6$ would close that. The GeoNames result — where the CPU's per-model early exit halves the batched trainer's advantage — motivates an adaptive batch schedule that periodically compacts the active set.

kNN. The accuracy fix comes first: replace the "stop at k candidates" rule with "grow until the k-th candidate's distance fits inside the box", which restores exactness at the cost of extra growth rounds (Section 6.5 quantifies how often: 57–59% of uniform queries currently stop short). Beyond that, the radius model needs help on skewed data: 300 training samples cannot represent GeoNames density, and the retry loop is where the time goes. Density-stratified sampling, more lattice nodes, or per-cell radius statistics are all cheap to try. The training-time brute-force radius computation also deserves a GPU port — it costs 147 s of single-core CPU time at 10^7, it is embarrassingly parallel, and until it moves we do not claim end-to-end GPU kNN acceleration, only the query path.

Updates. The sort-based reframing measures throughput, not a maintenance policy:

nothing re-balances shards or re-trains models as the distribution drifts. Pairing batched inserts with a shard-quality monitor and incremental re-training (the batched trainer makes re-training cheap) would turn the primitive into a usable dynamic index, and a comparison against ALEX-style in-place strategies [12] would situate it.

Local models. The MLP experiment's machinery invites better candidates: networks with explicit hinge structure, per-column hidden width chosen by column hardness, or simply more pieces (σ) in the existing model, which the batched trainer makes affordable. Any replacement must beat σ=50 piecewise-linear on *loss per byte and per evaluation nanosecond* — that is the bar Section 6.7 sets, and it is a high one.

Pipeline residency. Page generation — the build step that takes the trained models' predictions and physically groups the sorted points into shards and pages, producing the flat layout everything else reads — still runs on the CPU, and the build round-trips data between host and device once more than necessary as a result. A fully device-resident build (partition through flat layout without leaving the GPU) is mostly plumbing, and would matter for the re-training loop above.

Protocol note for re-runs. One small improvement for anyone repeating these benchmarks, which does not affect the conclusions drawn here: discarding (or reporting separately) each kernel's first invocation would keep one-time JIT compilation out of the small-N averages. In the present data this contamination is confined to the uniform run at N=10^5 and is flagged wherever it appears, with the warm-cache runs supplying the steady-state numbers at that size.

8. Conclusions

This dissertation set out to discover which parts of a complete learned spatial index — LISA's build, training, range queries, kNN and updates — benefit from GPU execution, and by how much. The method was to re-express every stage in batch-parallel form on top of the public reference implementation, keep CPU and GPU semantics aligned closely enough for paired correctness checks, and measure across three data distributions, three scales, four classical baselines and two GPUs.

The affirmative findings are concentrated where the pipeline is sort-shaped or embarrassingly batched. Build-phase partitioning and mapping collapse to single global sorts and gain one to two-plus orders of magnitude. Training all local models as one padded, masked, batched Newton problem — the thesis's main methodological contribution — yields 13–27× at $N{=}10^6$, turns 10^7-point training into a sub-three-minute task, and matches the sequential optimiser's fit on paired tests, while also exposing its limit: a batch is as slow as its slowest-converging member, which on hard real-world columns halves the win. The flat-candidate range-query formulation is the strongest result, 60–100× over the CPU implementation at scale and 11–17× ahead of a bulk-loaded R-tree on every distribution tested, with per-query costs of a few microseconds. Batched updates reach 17 million inserted points per second.

The negative findings are reported with equal weight, because they are where much of the transferable insight is. The monotonic MLP local model trains somewhat faster than the piecewise trainer and fits orders of magnitude worse on nearly every workload; it remains outside the query path. LISA's kNN, GPU or not, is far from competitive with a k-d tree at $d{=}2$ and in-memory scales, and a closer accuracy analysis shows its stopping rule returns approximate neighbour sets for over half the queries on uniform data — the GPU faithfully accelerates, four- to sevenfold, a design that the surrounding comparison shows to be the wrong one for this regime. And below 10^5 points, fixed GPU costs absorb most of the advantage.

In practical terms, the results support the following guidance: a GPU-resident LISA is a strong choice for read-heavy, batch-query workloads over millions of low-dimensional points — its range-query throughput and tiny learned footprint now come with a build cost that batched training has brought into the same band as a bulk-loaded R-tree's — and it is the wrong choice for point-at-a-time service, small datasets, or kNN-centric workloads in two dimensions. The path from here — tighter candidate analysis, an exact kNN stopping rule, adaptive training schedules and a real update policy — is laid out in Chapter 7, and all code, data, seeds and raw measurements accompany this thesis to make both the positive and the negative results checkable.

Bibliography

[1] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. The case for learned index structures. In *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*, pages 489–504, Houston, TX, USA, 2018. ACM.

[2] P. Li, H. Lu, Q. Zheng, L. Yang, and G. Pan. LISA: A learned index structure for spatial data. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2119–2133, Portland, OR, USA, 2020. ACM.

[3] V. Nathan, J. Ding, M. Alizadeh, and T. Kraska. Learning multi-dimensional indexes. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 985–1000, Portland, OR, USA, 2020. ACM.

[4] J. Liu, F. Zhang, L. Lu, C. Qi, X. Guo, D. Deng, G. Li, H. Zhang, J. Zhai, and X. Du. G-Learned Index: Enabling efficient learned index on GPU. *IEEE Transactions on Parallel and Distributed Systems*, 35(6):795–812, 2024.

[5] J. Kim, S. Hong, and B. Nam. A performance study of traversing spatial indexing structures in parallel on GPU. In *Proceedings of the 14th IEEE International Conference on High Performance Computing and Communication*, pages 900–907, 2012.

[6] A. Al-Mamun, H. Wu, Q. He, J. Wang, and W. G. Aref. A survey of learned indexes for the multi-dimensional space. *ACM Computing Surveys*, 58(4), 2026.

[7] A. Guttman. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, pages 47–57, Boston, MA, USA, 1984.

[8] R. A. Finkel and J. L. Bentley. Quad trees: A data structure for retrieval on composite keys. *Acta Informatica*, 4(1):1–9, 1974.

[9] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.

[10] S. T. Leutenegger, M. A. Lopez, and J. Edgington. STR: A simple and efficient algorithm for R-tree packing. In *Proceedings of the 13th International Conference on Data Engineering*, pages 497–506, 1997. IEEE.

[11] H. Wang, X. Fu, J. Xu, and H. Lu. Learned index for spatial queries. In *Proceedings of the 20th IEEE International Conference on Mobile Data Management*, pages 569–574, 2019.

[12] J. Ding, U. F. Minhas, J. Yu, C. Wang, J. Do, Y. Li, H. Zhang, B. Chandramouli,

J. Gehrke, D. Kossmann, D. Lomet, and T. Kraska. ALEX: An updatable adaptive learned index. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 969–984, Portland, OR, USA, 2020. ACM.

[13] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of the Workshop on Machine Learning Systems (LearningSys) at NIPS*, 2017.

[14] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.

[15] E. Garcia and M. Gupta. Lattice regression. In *Advances in Neural Information Processing Systems 22*, 2009.

[16] J. Sill. Monotonic networks. In *Advances in Neural Information Processing Systems 10*, 1997.

[17] P. Virtanen, R. Gommers, T. E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[18] GeoNames. The GeoNames geographical database. `https://www.geonames.org/`, accessed 2026.